MongoDB Practical Assignment - Set 1

Name:	Tharunkumar S	**Database:**	collegeDB
Department:	Artificial Intelligence & Data Science	**Collections:**	students, products
College:	RVS College of Engineering and Technology	**Questions:**	10

Queries were written in mongosh. ObjectId values will differ on your machine; all other output follows the sample data inserted in Question 1.

1. Create database collegeDB and collection students; insert at least 5 documents

Question: Create a database named collegeDB and a collection named students. Insert at least 5 student documents containing name, age, course, city and marks.

QUERY & OUTPUT

```
test> use collegeDB
switched to db collegeDB

collegeDB> db.createCollection("students")
{ ok: 1 }

collegeDB> db.students.insertMany([
  { name: 'Arun', age: 21, course: 'MERN', city: 'Bangalore', marks: 85 },
  { name: 'Priya', age: 22, course: 'Data Science', city: 'Mumbai', marks: 72 },
  { name: 'Karthik', age: 20, course: 'MERN', city: 'Chennai', marks: 65 },
  { name: 'Divya', age: 21, course: 'Java Full Stack', city: 'Bangalore', marks: 58 },
  { name: 'Rahul', age: 23, course: 'MERN', city: 'Mumbai', marks: 91 },
  { name: 'Sneha', age: 22, course: 'Data Science', city: 'Coimbatore', marks: 78 },
  { name: 'Vikram', age: 20, course: 'Python', city: 'Chennai', marks: 35 }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6720a1f4e8b2c9d3a5f10001'),
    '1': ObjectId('6720a1f4e8b2c9d3a5f10002'),
    '2': ObjectId('6720a1f4e8b2c9d3a5f10003'),
    '3': ObjectId('6720a1f4e8b2c9d3a5f10004'),
    '4': ObjectId('6720a1f4e8b2c9d3a5f10005'),
    '5': ObjectId('6720a1f4e8b2c9d3a5f10006'),
    '6': ObjectId('6720a1f4e8b2c9d3a5f10007')
  }
}

collegeDB> db.students.countDocuments()
7
```

2. Display all documents; then only students whose course is MERN

Question: Display all documents from students. Then display only students whose course is MERN.

QUERY & OUTPUT

```
collegeDB> db.students.find()
[
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10001'), name: 'Arun', age: 21, course: 'MERN', city: 'Bangalore', marks: 85 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10002'), name: 'Priya', age: 22, course: 'Data Science', city: 'Mumbai', marks: 72 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10003'), name: 'Karthik', age: 20, course: 'MERN', city: 'Chennai', marks: 65 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10004'), name: 'Divya', age: 21, course: 'Java Full Stack', city: 'Bangalore', marks: 58 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10005'), name: 'Rahul', age: 23, course: 'MERN', city: 'Mumbai', marks: 91 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10006'), name: 'Sneha', age: 22, course: 'Data Science', city: 'Coimbatore', marks: 78 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10007'), name: 'Vikram', age: 20, course: 'Python', city: 'Chennai', marks: 35 }
]

collegeDB> db.students.find({ course: "MERN" })
[
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10001'), name: 'Arun', age: 21, course: 'MERN', city: 'Bangalore', marks: 85 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10003'), name: 'Karthik', age: 20, course: 'MERN', city: 'Chennai', marks: 65 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10005'), name: 'Rahul', age: 23, course: 'MERN', city: 'Mumbai', marks: 91 }
]
```

3. Marks greater than 70; marks between 60 and 80

Question: Find all students whose marks are greater than 70. Then find students whose marks are between 60 and 80.

QUERY & OUTPUT

```
collegeDB> db.students.find({ marks: { $gt: 70 } })
[
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10001'), name: 'Arun', age: 21, course: 'MERN', city: 'Bangalore', marks: 85 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10002'), name: 'Priya', age: 22, course: 'Data Science', city: 'Mumbai', marks: 72 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10005'), name: 'Rahul', age: 23, course: 'MERN', city: 'Mumbai', marks: 91 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10006'), name: 'Sneha', age: 22, course: 'Data Science', city: 'Coimbatore', marks: 78 }
]

collegeDB> db.students.find({ marks: { $gte: 60, $lte: 80 } })
[
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10002'), name: 'Priya', age: 22, course: 'Data Science', city: 'Mumbai', marks: 72 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10003'), name: 'Karthik', age: 20, course: 'MERN', city: 'Chennai', marks: 65 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10006'), name: 'Sneha', age: 22, course: 'Data Science', city: 'Coimbatore', marks: 78 }
]
```

4. Students from Bangalore or Mumbai ($in operator)

Question: Find all students who belong to either Bangalore or Mumbai using an appropriate MongoDB operator.

QUERY & OUTPUT

```
collegeDB> db.students.find({ city: { $in: ["Bangalore", "Mumbai"] } })
[
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10001'), name: 'Arun', age: 21, course: 'MERN', city: 'Bangalore', marks: 85 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10002'), name: 'Priya', age: 22, course: 'Data Science', city: 'Mumbai', marks: 72 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10004'), name: 'Divya', age: 21, course: 'Java Full Stack', city: 'Bangalore', marks: 58 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10005'), name: 'Rahul', age: 23, course: 'MERN', city: 'Mumbai', marks: 91 }
]

// Same result using $or:
collegeDB> db.students.find({ $or: [ { city: "Bangalore" }, { city: "Mumbai" } ] })
```

5. updateOne() on marks; updateMany() on course for a city

Question: Update one student's marks using updateOne(). Then update the course of all students from a particular city using updateMany().

QUERY & OUTPUT

```
collegeDB> db.students.updateOne({ name: "Karthik" }, { $set: { marks: 70 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

collegeDB> db.students.updateMany({ city: "Chennai" }, { $set: { course: "Full Stack" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}

// Verify
collegeDB> db.students.find({ city: "Chennai" })
[
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10003'), name: 'Karthik', age: 20, course: 'Full Stack', city: 'Chennai', marks: 70 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10007'), name: 'Vikram', age: 20, course: 'Full Stack', city: 'Chennai', marks: 35 }
]
```

6. deleteOne(); deleteMany() for marks below 40

Question: Delete one student using deleteOne(). Then delete all students whose marks are below 40 using deleteMany().

QUERY & OUTPUT

```
collegeDB> db.students.deleteOne({ name: "Divya" })
{ acknowledged: true, deletedCount: 1 }

collegeDB> db.students.deleteMany({ marks: { $lt: 40 } })
{ acknowledged: true, deletedCount: 1 }

// Verify remaining documents
collegeDB> db.students.find({}, { _id: 0, name: 1, marks: 1 })
[
  { name: 'Arun', marks: 85 },
  { name: 'Priya', marks: 72 },
  { name: 'Karthik', marks: 70 },
  { name: 'Rahul', marks: 91 },
  { name: 'Sneha', marks: 78 }
]
```

7. products collection (8 products); find all products of one category

Question: Create a products collection and insert at least 8 products with name, category, price, stock and brand. Find all products from one category.

QUERY & OUTPUT

```
collegeDB> db.products.insertMany([
  { name: 'Laptop', category: 'Electronics', price: 55000, stock: 12, brand: 'Dell' },
  { name: 'Smartphone', category: 'Electronics', price: 18000, stock: 25, brand: 'Samsung' },
  { name: 'Headphones', category: 'Electronics', price: 1500, stock: 40, brand: 'Sony' },
  { name: 'Keyboard', category: 'Accessories', price: 900, stock: 60, brand: 'Logitech' },
  { name: 'Mouse', category: 'Accessories', price: 600, stock: 8, brand: 'HP' },
  { name: 'Office Chair', category: 'Furniture', price: 7500, stock: 6, brand: 'Godrej' },
  { name: 'Study Table', category: 'Furniture', price: 4200, stock: 15, brand: 'Nilkamal' },
  { name: 'Backpack', category: 'Accessories', price: 1200, stock: 30, brand: 'Wildcraft' }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6720a1f4e8b2c9d3a5f20001'),
    '1': ObjectId('6720a1f4e8b2c9d3a5f20002'),
    '2': ObjectId('6720a1f4e8b2c9d3a5f20003'),
    '3': ObjectId('6720a1f4e8b2c9d3a5f20004'),
    '4': ObjectId('6720a1f4e8b2c9d3a5f20005'),
    '5': ObjectId('6720a1f4e8b2c9d3a5f20006'),
    '6': ObjectId('6720a1f4e8b2c9d3a5f20007'),
    '7': ObjectId('6720a1f4e8b2c9d3a5f20008')
  }
}

collegeDB> db.products.find({ category: "Electronics" })
[
  { _id: ObjectId('6720a1f4e8b2c9d3a5f20001'), name: 'Laptop', category: 'Electronics', price: 55000, stock: 12, brand: 'Dell' },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f20002'), name: 'Smartphone', category: 'Electronics', price: 18000, stock: 25, brand: 'Samsung' },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f20003'), name: 'Headphones', category: 'Electronics', price: 1500, stock: 40, brand: 'Sony' }
]
```

8. Products with price > 1000 AND stock > 10

Question: Using products, find products with price greater than 1000 and stock greater than 10.

QUERY & OUTPUT

```
collegeDB> db.products.find({ price: { $gt: 1000 }, stock: { $gt: 10 } })
[
  { _id: ObjectId('6720a1f4e8b2c9d3a5f20001'), name: 'Laptop', category: 'Electronics', price: 55000, stock: 12, brand: 'Dell' },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f20002'), name: 'Smartphone', category: 'Electronics', price: 18000, stock: 25, brand: 'Samsung' },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f20003'), name: 'Headphones', category: 'Electronics', price: 1500, stock: 40, brand: 'Sony' },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f20007'), name: 'Study Table', category: 'Furniture', price: 4200, stock: 15, brand: 'Nilkamal' },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f20008'), name: 'Backpack', category: 'Accessories', price: 1200, stock: 30, brand: 'Wildcraft' }
]

// Same query written explicitly with $and:
collegeDB> db.products.find({ $and: [ { price: { $gt: 1000 } }, { stock: { $gt: 10 } } ] })
```

9. sort(), limit() and skip() on students

Question: Use sort(), limit(), and skip() on students. Sort by marks, display the top 3 students, and skip the first 2 documents.

QUERY & OUTPUT

```
// Top 3 students (highest marks first)
collegeDB> db.students.find().sort({ marks: -1 }).limit(3)
[
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10005'), name: 'Rahul', age: 23, course: 'MERN', city: 'Mumbai', marks: 91 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10001'), name: 'Arun', age: 21, course: 'MERN', city: 'Bangalore', marks: 85 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10006'), name: 'Sneha', age: 22, course: 'Data Science', city: 'Coimbatore', marks: 78 }
]

// Skip the first 2 documents of the sorted result
collegeDB> db.students.find().sort({ marks: -1 }).skip(2)
[
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10006'), name: 'Sneha', age: 22, course: 'Data Science', city: 'Coimbatore', marks: 78 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10002'), name: 'Priya', age: 22, course: 'Data Science', city: 'Mumbai', marks: 72 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10003'), name: 'Karthik', age: 20, course: 'MERN', city: 'Chennai', marks: 70 }
]

// sort + skip + limit together
collegeDB> db.students.find().sort({ marks: -1 }).skip(2).limit(3)
[
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10006'), name: 'Sneha', age: 22, course: 'Data Science', city: 'Coimbatore', marks: 78 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10002'), name: 'Priya', age: 22, course: 'Data Science', city: 'Mumbai', marks: 72 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10003'), name: 'Karthik', age: 20, course: 'MERN', city: 'Chennai', marks: 70 }
]
```

10. find() vs findOne(); updateOne() vs updateMany()

Question: Write and demonstrate the difference between find() and findOne(), and between updateOne() and updateMany().

QUERY & OUTPUT

```
// ---------- find() vs findOne() ----------
collegeDB> db.students.find({ course: "MERN" })
[
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10001'), name: 'Arun', age: 21, course: 'MERN', city: 'Bangalore', marks: 85 },
  { _id: ObjectId('6720a1f4e8b2c9d3a5f10005'), name: 'Rahul', age: 23, course: 'MERN', city: 'Mumbai', marks: 91 }
]

collegeDB> db.students.findOne({ course: "MERN" })
{ _id: ObjectId('6720a1f4e8b2c9d3a5f10001'), name: 'Arun', age: 21, course: 'MERN', city: 'Bangalore', marks: 85 }

// ---------- updateOne() vs updateMany() ----------
collegeDB> db.students.updateOne({ city: "Mumbai" }, { $set: { status: "Active" } })
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }

collegeDB> db.students.updateMany({ city: "Mumbai" }, { $set: { status: "Passed" } })
{ acknowledged: true, insertedId: null, matchedCount: 2, modifiedCount: 2, upsertedCount: 0 }
```

Difference summary (Q10)

	Method 1	Method 2
Read	find(): returns a cursor with ALL matching documents (empty cursor if none)	findOne(): returns only the FIRST matching document (null if none)
Update	updateOne(): modifies only the FIRST matching document	updateMany(): modifies ALL matching documents